

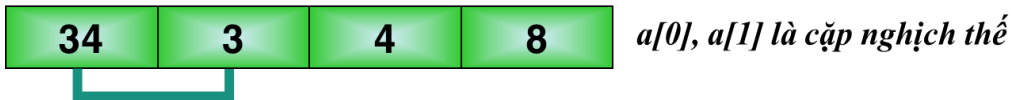
CÁC GIẢI THUẬT SẮP XẾP

3.1 Bài toán sắp xếp

- Sắp xếp là quá trình bố trí lại các phần tử của một tập đối tượng nào đó theo một thứ tự nhất định thỏa mãn một tiêu chuẩn nào đó dựa trên nội dung thông tin lưu giữ tại mỗi phần tử.
- Yêu cầu sắp xếp thường xuyên xuất hiện trong các ứng dụng tin học với các mục đích khác nhau: sắp xếp dữ liệu trong máy tính để tìm kiếm cho thuận lợi, sắp xếp các kết quả xử lý để in ra trên bảng biểu...
- Có nhiều giải thuật sắp xếp: Selection sort, Insertion sort, Interchange sort, Bubble sort, Shaker sort, Binary Insertion sort, Shell sort, Heap sort, Quick sort, Merge sort, Radix sort...
- Các nội dung sau đây sẽ trình bày theo hướng sắp xếp các phần tử tăng dần.*

3.2 Đổi chỗ trực tiếp - Interchange Sort

- Ý tưởng:** Xuất phát từ đầu dãy, tìm tất cả các **nghịch thế** chứa phần tử này (hai phần tử gọi là nghịch thế nếu khi sắp xếp tăng dần mà phần tử trước có giá trị lớn hơn phần tử đứng sau), **triệt tiêu chúng bằng cách đổi chỗ 2 phần tử trong cặp nghịch thế**. Lặp lại xử lý trên với phần tử kế trong dãy.



- Các bước tiến hành**
 - Bước 1: $i = 0$; // bắt đầu từ đầu dãy
 - Bước 2: $j = i + 1$; // tìm các nghịch thế với $a[i]$
 - Bước 3:
 - Trong khi $j < N$ thực hiện
 - Nếu $a[j] < a[i]$ // xét cặp $a[i], a[j]$
 - đổi chỗ ($a[i], a[j]$);
 - $j = j + 1$;
 - Bước 4: $i = i + 1$;
 - Nếu $i < N - 1$: Lặp lại Bước 2.
 - Ngược lại: Dừng.
- Ví dụ:** Minh họa sắp xếp dãy số: 1, 4, 8, 9, 0, 12, -5 tăng dần bằng giải thuật đổi chỗ trực tiếp.

- Cài đặt thuật toán

```
void InterchangeSort(int a[], int N)
{
    int i, j;
    for(i=0; i<N-1; i++) {
        for(j=i+1; j<N; j++) {
            if(a[j]<a[i]) // Thỏa mãn là cặp nghịch thế
            { // Đổi chỗ - có thể tách thành một hàm riêng
                int t=a[i]; //biến tạm
                a[i]=a[j];
                a[j]=t;
            }
        }
    }
}
```

- Độ phức tạp thuật toán:

- In general, we **compare keys and move items** (or exchange items) in a sorting algorithm (which uses key comparisons).
- So, to analyze a sorting algorithm we should **count the number of key comparisons and the number of moves**.
- Ignoring other operations does not affect our final result.

Trường hợp	Số lần so sánh	Số lần hoán vị
Tốt nhất	$\frac{n(n-1)}{2}$	0
Xấu nhất	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$

✓ Số lần so sánh

- Bắt buộc phải so sánh từng cặp (dù mảng đã được sắp xếp hay chưa).
- Công thức:

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

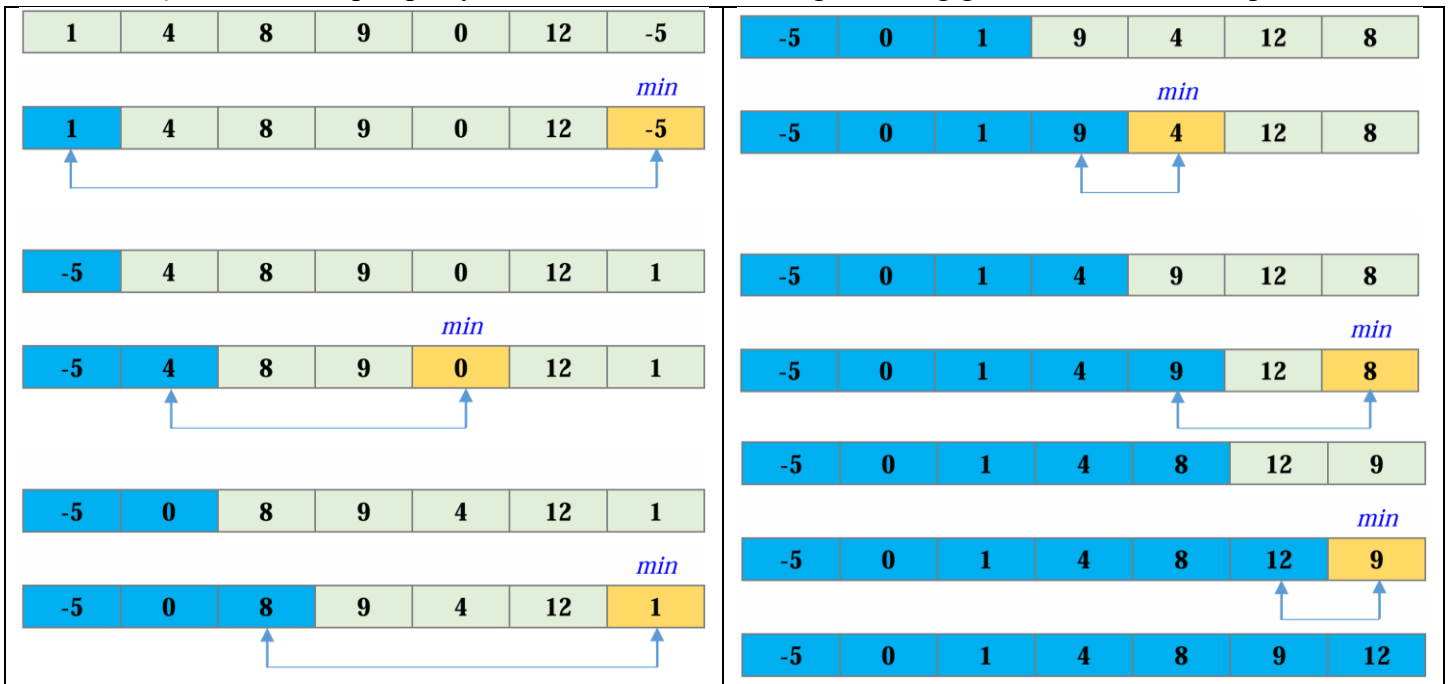
→ Đây là cấp số cộng, kết quả $\sim O(n^2)$.

✓ Số lần hoán vị

- Trường hợp tốt nhất (đã sắp xếp): không cần hoán vị → 0 lần.
- Trường hợp xấu nhất (đảo ngược hoàn toàn): mỗi lần so sánh đều cần đổi chỗ → tối đa $\frac{n(n-1)}{2}$ lần.

3.3 Chọn trực tiếp - Selection Sort

- **Ý tưởng:**
 - **Chọn phần tử nhỏ nhất** trong N phần tử trong dãy hiện hành ban đầu và **đưa phần tử này về vị trí đầu dãy** hiện hành.
 - Xem dãy hiện hành chỉ còn N-1 phần tử của dãy hiện hành ban đầu. Bắt đầu từ vị trí thứ 2, **chọn phần tử nhỏ nhất trong số N-1 phần tử** kể trên và đưa nó về vị trí đầu của dãy gồm N-1 phần tử đang xét.
 - Lặp lại quá trình trên cho dãy hiện hành... **đến khi dãy hiện hành chỉ còn 1 phần tử.**
- **Các bước thực hiện:**
 - Bước 1: $i = 0$;
 - Bước 2: Tìm phần tử **a[min]** nhỏ nhất trong dãy hiện hành từ $a[i]$ đến $a[N]$
 - Bước 3: Đổi chỗ $a[\text{min}]$ và $a[i]$
 - Bước 4:
 - Nếu $i < N-1$ thì $i = i+1$; Lặp lại Bước 2;
 - Ngược lại: Dừng.
- **Ví dụ:** Minh họa sắp xếp dãy số: 1, 4, 8, 9, 0, 12, -5 tăng dần bằng giải thuật chọn trực tiếp.



- **Cài đặt thuật toán**

```
void SelectionSort(int a[], int n) {
    int min, i, j; // min dùng để lưu vị trí của phần tử nhỏ nhất

    for (i = 0; i < n - 1; i++) {
        min = i;

        // Tìm phần tử nhỏ nhất trong đoạn chưa sắp xếp
        for (j = i + 1; j < n; j++) {
            if (a[j] < a[min]) {
                min = j;
            }
        }

        // Hoán đổi phần tử nhỏ nhất với phần tử đầu tiên của đoạn chưa sắp xếp
        if (min != i) {
            int t = a[i];
            a[i] = a[min];
            a[min] = t;
        }
    }
}
```

- **Độ phức tạp thuật toán:**

- The outer for loop executes $n-1$ times.
 - We invoke swap function once at each iteration.
 - Total Swaps: $n-1$
 - Total Moves: $3*(n-1)$ (Each swap has three moves)
- The inner for loop executes the size of the unsorted part minus 1 (from 1 to $n-1$), and in each iteration we make one key comparison.
 - # of key comparisons = $1+2+ \dots +n-1 = [n*(n-1)]/2$
 - **So, Selection sort is $O(n^2)$**
- The best case, the worst case, and the average case of the selection sort algorithm are same → All of them are $O(n^2)$
 - This means that the behavior of the selection sort algorithm does not depend on the initial organization of data.
 - Since $O(n^2)$ grows so rapidly, **the selection sort algorithm is appropriate only for small n .**
 - **Although the selection sort algorithm requires $O(n^2)$ key comparisons, it only requires $O(n)$ moves.**
 - A selection sort could be a good choice **if data moves are costly but key comparisons are not costly** (short keys, long records) → Selection Sort giảm số lần hoán vị, tiết kiệm chi phí tổng thể.

3.4 Chèn trực tiếp - Insertion Sort

- **Ý tưởng:**
 - Virtually *split the array into a sorted and an unsorted part*
 - **Assume that the first element is already sorted** and remaining elements are unsorted
 - Select an unsorted element and *compare with all elements in the sorted part*
 - **If element(s) in the sorted part is smaller than the selected element**, proceed to the next element in the unsorted part. **Else**, *shift larger elements in the sorted part towards the right*, insert the selected element at the right index
 - Repeat till all the unsorted elements are placed in the right order

- **Các bước thực hiện**

- Bước 1: **$i = 1$** ; //giả sử có đoạn $a[0]$ đã được sắp xếp
- Bước 2: **$x = a[i]$** ; Tìm vị trí position thích hợp trong đoạn $a[1]$ đến $a[i-1]$ để chèn $a[i]$ vào
- Bước 3: Dời chỗ các phần tử **từ $a[\text{position}]$ đến $a[i-1]$** sang phải 1 vị trí để dành chỗ cho $a[i]$
- Bước 4: **$a[\text{position}] = x$** ; //có đoạn $a[1] \dots a[i]$ đã được sắp xếp
- Bước 5: **$i = i+1$** ;

Nếu $i < n$: Lặp lại Bước 2

Ngược lại: Dừng

- **Ví dụ:** (NTI = Number to insert, SE = Sorted element)

[-6 20 8 -2 4]	NTI = 20	SE = -6	-6 > 20? No. Place 20 to the right of -6
[-6 20 8 -2 4]	NTI = 8	SE = 20	20 > 8? Yes. Shift 20 to the right
[-6 20 20 -2 4]	NTI = 8	SE = -6	-6 > 8? No. Place 8 to the right of -6
[-6 8 20 -2 4]	NTI = -2	SE = 20	20 > -2? Yes. Shift 20 to the right
[-6 8 20 20 4]	NTI = -2	SE = 8	8 > -2? Yes. Shift 8 to the right
[-6 8 8 20 4]	NTI = -2	SE = -6	-6 > 2? No. Place -2 to the right of -6
[-6 -2 8 20 4]	NTI = 4	SE = 20	20 > 4? Yes. Shift 20 to the right
[-6 -2 8 20 20]	NTI = 4	SE = 8	8 > 4? Yes. Shift 8 to the right
[-6 -2 8 8 20]	NTI = 4	SE = -2	-2 > 4? No. Place -4 to the right of -2
[-6 -2 4 8 20]			Reached end of array. Array is sorted.

- Cài đặt thuật toán

```
void InsertionSort(int a[], int n)    //n là chiều dài của mảng
{
    int pos, i, numberToInsert;
    for(i=1; i<n; i++)
    {
        numberToInsert = a[i];           // (1) copy/key move
        //lưu giá trị a[i] tránh bị ghi đè khi dời chỗ các phần tử.
        pos = i - 1;
        /*biến dùng để xác định vị trí cần chèn numberToInsert & so sánh giữa phần tử cần
        chèn và các phần tử trong mảng đã sắp xếp hiện hành*/
        while(pos >= 0 && a[pos] > numberToInsert)
        {
            a[pos+1] = a[pos];           // (2) shift move
            pos--;
        }
        a[pos+1] = numberToInsert;       // (3) insert move
    }
}
```

- Độ phức tạp thuật toán:

- Running time depends on *not only the size of the array but also the contents of the array.*
 - Số vòng ngoài luôn là $n-1$ (i chạy từ 1 đến $n-1$) — phụ thuộc kích thước.
 - Nhưng **số lần lặp trong vòng while** phụ thuộc vào **mức độ “không sắp xếp” – nội dung** của mảng:
 - Nếu mảng đã sắp xếp (tốt nhất) → inner loop hiếm khi chạy.
 - Nếu mảng đảo ngược (xấu nhất) → inner loop chạy nhiều nhất (phải dời mọi phần tử lớn hơn).
- **Best-case → $O(n)$**
 - Array is already sorted in ascending order.
 - Inner loop will not be executed.
 - The number of moves: $2 \cdot (n-1)$
 - *move = một phép gán (ví dụ $key = A[i]$, $A[pos+1] = A[pos]$, $A[pos+1] = numberToInsert$).*
 - Trong TH best-case, với mỗi i có 2 phép gán: $numberToInsert = A[i]$ và $A[pos+1] = numberToInsert \rightarrow$ tổng $2 \cdot (n-1)$.
 - The number of key comparisons: $(n-1)$ [mỗi i thực hiện 1 so sánh $A[j] > key$]
- **Worst-case → $O(n^2)$**
 - Array is in reverse order.
 - Inner loop is executed $n-1$ times, for $i = 1, 2, \dots, n-1$
 - The number of moves: $2 \cdot (n-1) + (1+2+ \dots + n-1) = 2 \cdot (n-1) + n \cdot (n-1)/2$
 - 2 moves cho mỗi i như ở best-case (copy key + insert) → $2 \cdot (n-1)$.
 - Thêm mỗi shift là 1 move → tổng shifts = $n \cdot (n-1)/2$

Ví dụ $n = 5$

Mảng giảm dần: [5,4,3,2,1]

- $i = 2$: phần tử 4 so với [5] → dịch 1 phần tử.
- $i = 3$: phần tử 3 so với [5,4] → dịch 2 phần tử.
- $i = 4$: phần tử 2 so với [5,4,3] → dịch 3 phần tử.
- $i = 5$: phần tử 1 so với [5,4,3,2] → dịch 4 phần tử.

$$\text{Tổng số dịch} = 1 + 2 + 3 + 4 = 10 = \frac{5 \cdot 4}{2}.$$

- The number of key comparisons:
 - Nếu chỉ đếm các so sánh thành công ($A[j] > \text{key}$) trong inner loop thì số đó là:

$$1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2}.$$

- Nếu bạn đếm cả phép so sánh thất bại cuối cùng trong mỗi vòng (khi while dừng), thì phải cộng thêm $(n - 1)$ (một phép so sánh thất bại cho mỗi i), dẫn tới

$$\frac{n(n - 1)}{2} + (n - 1) = \frac{n(n + 1)}{2} - 1.$$

(Vì vậy các sách khác nhau có thể báo số so sánh hơi khác nhau do qui ước đếm.)

- **Average-case $\rightarrow O(n^2)$**

- We have to look at all possible initial data organizations.

- **So, Insertion Sort is $O(n^2)$**

3.5 Nổi bọt - Bubble Sort

- Ý tưởng:

- **Xuất phát từ cuối dãy, đổi chỗ các cặp phần tử kế cận để đưa phần tử nhỏ hơn trong cặp phần tử đó về vị trí đứng đầu dãy hiện hành**, sau đó sẽ **không xét đến nó ở bước tiếp theo**, do vậy ở lần xử lý thứ i , nó sẽ đứng ở vị trí đầu dãy là i .
 - Thông thường, thuật toán này thực hiện theo **hướng ngược lại**, được mô tả như sau: **Xuất phát từ đầu dãy**, so sánh và đổi chỗ các cặp phần tử kế cận để đưa **phần tử lớn hơn trong cặp** về vị trí cuối dãy hiện hành. Sau đó, phần tử này không cần xét lại ở các bước tiếp theo. Vì vậy, ở lần xử lý thứ i , **vị trí cuối dãy** sẽ là phần tử thứ $n - i + 1$ (tính từ 1) (hoặc $n - i - 1$ nếu tính từ 0), chỉ cần xét đến phần tử đó trở về trước.
- Lặp lại xử lý trên cho đến khi không còn cặp phần tử nào để xét.

- Cài đặt thuật toán

Xuất phát từ đầu mảng	Xuất phát từ cuối mảng
<pre> void BubbleSort(int a[], int n) { int i, j; bool swapped; for (i = 0; i < n-1; i++) { swapped = false; for (j = 0; j < n-i-1; j++) { if(a[j] > a[j+1]) { swap(a[j], a[j+1]); swapped = true; } } //Không còn phần tử để swap //=> mảng đã sắp xếp xong if (swapped == false) break; } } </pre>	<pre> void BubbleSort(int a[], int n) { int i, j; bool swapped; for (i = 0; i < n-1; i++) { swapped = false; for (j = n-1; j > i; j--) { if(a[j] < a[j-1]) { swap(a[j], a[j-1]); swapped = true; } } //Không còn phần tử để swap //=> mảng đã sắp xếp xong if (swapped == false) break; } } </pre>

- Ví dụ:

Xuất phát từ đầu mảng	Xuất phát từ cuối mảng					
<table><tr><td>6</td><td>4</td><td>1</td><td>2</td><td>5</td></tr></table>	6	4	1	2	5	
6	4	1	2	5		
Iteration 1:	Iteration 2:					
6 4 1 2 5	4 1 2 5 6					
4 6 1 2 5	1 4 2 5 6					
4 1 6 2 5	1 2 4 5 6					
4 1 2 6 5	1 2 4 5 6					
4 1 2 5 6						

<i>i</i>						<i>j</i>
1	4	8	9	0	12	-5
<i>i</i>						<i>j</i>
1	4	8	9	0	-5	12
<i>i</i>						<i>j</i>
1	4	8	9	0	-5	12
<i>i</i>						<i>j</i>
1	4	8	9	-5	0	12
<i>i</i>						<i>j</i>
1	4	8	9	-5	0	12
<i>i</i>						<i>j</i>
1	4	8	-5	9	0	12

Iteration 3: Iteration 4:

1 2 4 5 6 **1 2** 4 5 6

1 **2 4** 5 6 1 2 4 5 6

1 2 4 5 6

* Thực ra thuật toán đã dừng sau vòng lặp thứ 3, vì lúc đó biến *swapped* = *false*, nghĩa là mảng đã được sắp xếp hoàn tất.

<i>i</i>			<i>j</i>			
1	4	8	-5	9	0	12
<i>i</i>			<i>j</i>			
1	4	-5	8	9	0	12
<i>i</i>		<i>j</i>				
1	4	-5	8	9	0	12
<i>i</i>	<i>j</i>					
1	-5	4	8	9	0	12
<i>i</i>	<i>j</i>					
1	-5	4	8	9	0	12
-5	1	4	8	9	0	12
<i>i</i>			<i>j</i>			
-5	1	4	8	9	0	12
<i>i</i>		<i>j</i>				
-5	1	4	8	9	0	12
<i>i</i>	<i>j</i>					
-5	1	4	8	0	9	12
<i>i</i>		<i>j</i>				
-5	1	4	8	0	9	12
<i>i</i>	<i>j</i>					
-5	1	4	0	8	9	12
<i>i</i>		<i>j</i>				
-5	1	4	0	8	9	12
<i>i</i>	<i>j</i>					
-5	1	0	4	8	9	12
<i>i</i>	<i>j</i>					
-5	1	0	4	8	9	12
<i>i</i>	<i>j</i>					
-5	0	1	4	8	9	12
<i>i</i>		<i>j</i>				
-5	0	1	4	8	9	12
<i>i</i>	<i>j</i>					
-5	0	1	4	8	9	12
<i>i</i>	<i>j</i>					
-5	0	1	4	8	9	12

			<i>i</i>	<i>j</i>		
-5	0	1	4	8	9	12
			<i>i</i>		<i>j</i>	
-5	0	1	4	8	9	12
			<i>i</i>	<i>j</i>		
-5	0	1	4	8	9	12
			<i>i</i>	<i>j</i>		
-5	0	1	4	8	9	12
-5	0	1	4	8	9	12

• **Độ phức tạp thuật toán:**

- **Best-case:** $O(n)$
 - Array is already sorted in ascending order.
 - The number of moves: $0 \rightarrow O(1)$
 - The number of key comparisons: $(n-1) \rightarrow O(n)$
- **Worst-case:** $O(n^2)$
 - Array is in reverse order
 - Outer loop is executed $n-1$ times,
 - The number of moves: $3 \cdot (1+2+ \dots +n-1) = 3 \cdot n \cdot (n-1)/2 \rightarrow O(n^2)$
 - The number of key comparisons: $(1+2+ \dots +n-1) = n \cdot (n-1)/2 \rightarrow O(n^2)$
- **Average-case:** $O(n^2)$
 - We have to look at all possible initial data organizations.

○ **So, Bubble Sort is $O(n^2)$.**

3.6 Quick Sort – Sắp xếp nhanh, hay Partition Sort – Sắp xếp kiểu phân đoạn

- Quicksort is based on the **divide-and-conquer paradigm** (*Chia bài toán lớn thành những bài toán nhỏ hơn, và tiến hành giải các bài toán con đó*).
- Gọi là Quick Sort vì nó được đánh giá là ưu việt hơn các thuật toán sort cơ bản đã trình bày trước đó.
- **Ý tưởng:**
 - QuickSort chia mảng thành hai danh sách bằng cách so sánh từng phần tử của danh sách với một phần tử được chọn được gọi là phần tử chốt (pivot).
 - **Nhiệm vụ chính của thuật toán này là đưa pivot về đúng vị trí trong mảng, nghĩa là các phần tử nhỏ hơn pivot sẽ nằm bên trái nó, và các phần tử lớn hơn (hoặc bằng) pivot sẽ nằm bên phải nó.**
 - Tiến hành đưa **những phần tử nhỏ hơn phần tử chốt** được đưa về phía trước và nằm trong danh sách con thứ nhất, các **phần tử lớn hơn hoặc bằng phần tử chốt** được đưa về phía sau và thuộc danh sách con thứ hai.
 - Cứ tiếp tục chia các danh sách con như vậy tới khi **các danh sách con đều có độ dài bằng 1**.



- **Chọn Pivot**

- Chọn phần tử ngẫu nhiên/ vị trí giữa, đầu tiên, cuối cùng
 - **The popular, uninformed choice is to use the first element as the pivot.** This is acceptable if the input is random, *but if the input is presorted (or input with a large presorted section) or in reverse order, then the pivot provides a poor partition*, because either all the elements go into part 1 or they go into part 2.
- Phần tử trung vị (Median) hay *Median-of-Three* (chọn một trong 3 phần tử nằm ở đầu-giữa-cuối của mảng chưa sắp xếp, và phần tử được chọn có **giá trị** nằm giữa giá trị của 2 phần tử còn lại).

- **Các bước thực hiện**

- **Bước 1:** Lấy 1 phần tử bất kì của dãy làm **phần tử chốt X**.
- **Bước 2:** Tiến hành **duyệt từ bên trái** dãy cho đến khi gặp phần tử **lớn hơn hoặc bằng X** thì dừng. Đồng thời **duyệt từ bên phải** dãy cho đến khi gặp phần tử **nhỏ hơn hoặc bằng X** thì dừng.
- **Bước 3:** **Đổi chỗ 2 phần tử tìm được ở bước 2.**
- **Bước 4:** Tiếp tục duyệt cho đến khi **2 biến duyệt từ 2 chiều gặp nhau**. Khi đó dãy ban đầu đã phân thành 2 phần: phần trái gồm những phần tử nhỏ hơn X, phần phải gồm những phần tử lớn hơn hoặc bằng X.
- **Bước 5:** **Lặp lại quá trình phân hoạch đối với phần trái và phần phải mới** được tạo ở trên cho đến khi nào dãy được sắp xếp hoàn toàn.

- **Cài đặt thuật toán**

```
void QuickSort(int a[], int left, int right) {
    int i, j, pivot;
    pivot = a[(left + right) / 2];
    i = left; j = right;
    while (i <= j) {
        while (a[i] < pivot) i++;
        while (a[j] > pivot) j--;
        if (i <= j) {
            Swap(a[i], a[j]);
            i++; j--;
        }
    }
    if (left < j)    QuickSort(a, left, j);
    if (i < right)  QuickSort(a, i, right);
}

void QuickSort(int a[], int n) {
    QuickSort(a, 0, n - 1);
}
```

- Pivot được chọn là giá trị của phần tử ở vị trí giữa của đoạn dãy, tính bằng $(left + right) / 2$. Đây là một chiến lược phổ biến để chọn pivot, giúp giảm khả năng gặp trường hợp xấu nhất (khi dãy đã gần sắp xếp hoặc sắp xếp ngược).
- **Vòng lặp while(a[i] < pivot):** Di chuyển con trỏ i từ trái sang phải cho đến khi tìm được phần tử đầu tiên có giá trị **lớn hơn hoặc bằng** pivot. Điều này đảm bảo rằng các phần tử trước i đều nhỏ hơn pivot.
- **Vòng lặp while(a[j] > pivot):** Di chuyển con trỏ j từ phải sang trái cho đến khi tìm được phần tử đầu tiên có giá trị **nhỏ hơn hoặc bằng** pivot. Điều này đảm bảo rằng các phần tử sau j đều lớn hơn pivot.

- **Kiểm tra if($i \leq j$):** Nếu i và j chưa vượt qua nhau ($i \leq j$), đổi chỗ hai phần tử $a[i]$ và $a[j]$ để đưa phần tử lớn hơn về phía bên phải và phần tử nhỏ hơn về phía bên trái. Sau đó:
 - Tăng i lên 1 để tiếp tục xét phần tử tiếp theo bên trái.
 - Giảm j xuống 1 để xét phần tử tiếp theo bên phải.
- **Điều kiện dừng while($i < j$):** Vòng lặp phân hoạch dừng khi i và j vượt qua nhau ($i \geq j$). Lúc này, đoạn dãy đã được phân hoạch thành hai phần:
 - *Từ left đến j:* Các phần tử nhỏ hơn hoặc bằng pivot.
 - *Từ i đến right:* Các phần tử lớn hơn hoặc bằng pivot.

Hai đoạn này có thể chưa được sắp xếp, vì thế ta cần thực hiện gọi đệ quy hàm QuickSort cho 2 đoạn dãy con này.
- Điều kiện **left < j** và **i < right** đảm bảo chỉ gọi đệ quy khi đoạn dãy còn ít nhất hai phần tử. Nếu đoạn chỉ có một phần tử hoặc rỗng, thuật toán không cần gọi đệ quy.
 - Nếu **left < j**, gọi đệ quy **QuickSort(a, left, j)** để sắp xếp đoạn bên trái.
 - Nếu **i < right**, gọi đệ quy **QuickSort(a, i, right)** để sắp xếp đoạn bên phải.
- **Ví dụ:** Xét mảng $a = [3, 8, \underline{2}, 5, 1, 4]$, gọi QuickSort(a, 0, 5) (tính chỉ số từ 0):
 - **Bước 1:**
 - Chọn **pivot** = $a[(0+5)/2] = a[2] = 2$.
 - $i = 0, j = 5$.
 - **Phân hoạch:**
 - while($a[i] < 2$): $a[0] = 3 < 2 \rightarrow$ False, dừng, khi đó **i = 0**.
 - while($a[j] > 2$): $a[5] = 4 > 2 \rightarrow$ True, giảm $j--$
 - $j = 4$; while($a[j] > 2$): $a[4] = 1 > 2 \rightarrow$ False, dừng, khi đó **j = 4**.
 - Kiểm tra $i \leq j$ ($0 \leq 4$) $\rightarrow$ True, đổi chỗ $a[0] = 3$ và $a[4] = 1$.
 - Mảng thành **[1, 8, 2, 5, 3, 4]**. Tăng **i = 1**, giảm **j = 3**.
 - **Tiếp tục:**
 - while($a[i] < 2$): $a[1] = 8 < 2 \rightarrow$ False, dừng, khi đó **i = 1**;
 - while($a[j] > 2$): $a[3] = 5 > 2 \rightarrow$ True, giảm $j--$
 - $j = 2$; while($a[j] > 2$): $a[2] = 2 > 2 \rightarrow$ False, dừng, khi đó **j = 2**.
 - Kiểm tra $i \leq j$ ($1 \leq 2$), đổi chỗ $a[1] = 8$ và $a[2] = 2$.
 - Mảng thành **[1, 2, 8, 5, 3, 4]**. Tăng **i = 2**, giảm **j = 1**.
 - $i > j$ ($2 > 1$), dừng phân hoạch.
 - **Kết quả:**
 - Mảng **[1, 2, 8, 5, 3, 4]**, với $j = 1, i = 2$.
 - Các phần tử có chỉ số từ 0 đến 1 ([1, 2]) nhỏ hơn hoặc bằng pivot, từ 2 đến 5 ([8, 5, 3, 4]) lớn hơn hoặc bằng pivot.
 - **Bước 2:**
 - Gọi đệ quy QuickSort(a, 0, 1): Sắp xếp [1, 2].
 - Gọi đệ quy QuickSort(a, 2, 5): Sắp xếp [8, 5, 3, 4].
 - Các bước đệ quy tiếp tục tương tự, cuối cùng cho mảng sắp xếp tăng dần: **[1, 2, 3, 4, 5, 8]**.
- **Độ phức tạp thuật toán:**
 - **Trường hợp tốt nhất: $O(n \log n)$**
 - **Điều kiện:** Pivot chia dãy thành hai phần có kích thước gần bằng nhau (tức là mỗi phần có khoảng $n/2$ phần tử).

- *Lý do:*
 - Độ sâu của cây đệ quy là $O(\log n)$, vì mỗi lần gọi đệ quy giảm kích thước dãy xuống một nửa.
 - Mỗi lần phân hoạch tốn $O(n)$ phép so sánh để duyệt qua toàn bộ đoạn dãy.
 - Tổng: $O(n \log n)$.
- **Trường hợp trung bình: $O(n \log n)$**
 - Pivot không nhất thiết chia dãy thành hai phần bằng nhau, nhưng trung bình, các phân hoạch tạo ra các đoạn con có kích thước không quá chênh lệch.
- **Trường hợp xấu nhất: $O(n^2)$**
 - The worst case depends on strategy for choosing pivot: **leftmost (or rightmost) element is chosen as pivot**, the worst occurs in following cases.
 - *Array is already sorted in same order.*
 - Giả sử mảng đã được sắp xếp tăng dần (ví dụ: [1, 2, 3, 4, 5]), và pivot được chọn là phần tử đầu tiên ($a[\text{left}]$).
 - Khi chọn pivot là phần tử đầu tiên (ví dụ: 1 trong [1, 2, 3, 4, 5]), **tất cả các phần tử còn lại đều lớn hơn hoặc bằng pivot.**
 - *Kết quả phân hoạch:*
 - Phần bên trái (nhỏ hơn pivot): rỗng (vì không có phần tử nào nhỏ hơn 1).
 - Phần bên phải (lớn hơn hoặc bằng pivot): chứa tất cả các phần tử còn lại ([2, 3, 4, 5]).
 - Sau lần phân hoạch đầu tiên, đoạn bên phải có kích thước $n-1$, và thuật toán gọi đệ quy trên đoạn này.
 - Tiếp tục, ở lần phân hoạch tiếp theo, pivot lại là phần tử đầu tiên của đoạn con (ví dụ: 2), và quá trình lặp lại, chỉ loại bỏ một phần tử mỗi lần.
 - **Hậu quả:** Cần n lần phân hoạch, mỗi lần thực hiện khoảng $n, n-1, n-2, \dots, 1$ phép so sánh, dẫn đến độ phức tạp $O(n^2)$.
 - *Array is already sorted in reverse order.*
 - Giả sử mảng đã được sắp xếp giảm dần (ví dụ: [5, 4, 3, 2, 1]), và pivot được chọn là phần tử đầu tiên ($a[\text{left}]$).
 - Khi chọn pivot là phần tử đầu tiên (ví dụ: 5), **tất cả các phần tử còn lại đều nhỏ hơn pivot.**
 - *Kết quả phân hoạch:*
 - Phần bên trái (nhỏ hơn pivot): chứa tất cả các phần tử còn lại ([4, 3, 2, 1]).
 - Phần bên phải (lớn hơn hoặc bằng pivot): rỗng (vì không có phần tử nào lớn hơn hoặc bằng 5).
 - Sau lần phân hoạch đầu tiên, đoạn bên trái có kích thước $n-1$, và thuật toán gọi đệ quy trên đoạn này. Quá trình lặp lại, mỗi lần chỉ loại bỏ một phần tử (pivot).
 - **Hậu quả:** Tương tự trường hợp 1, cần n lần phân hoạch, mỗi lần thực hiện khoảng $n, n-1, \dots, 1$ phép so sánh, dẫn đến độ phức tạp $O(n^2)$.

- *All elements are same (special case of case 1 and 2)*

- Mỗi lần phân hoạch chỉ loại bỏ một phần tử, tương tự trường hợp 1 → cần n lần phân hoạch, dẫn đến độ phức tạp $O(n^2)$.

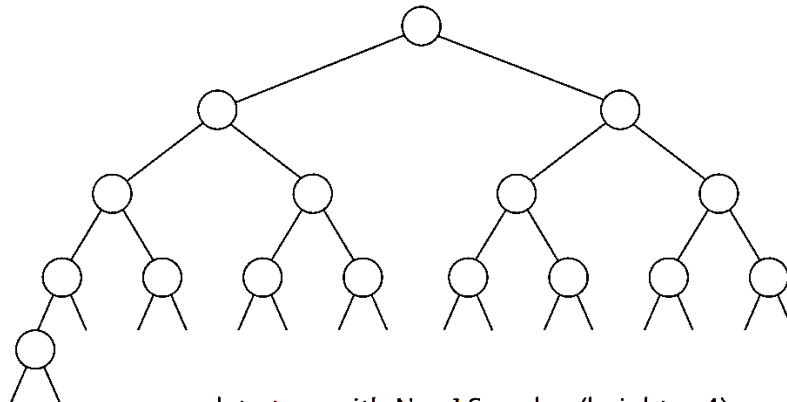
- **Nhận xét:**

- Quicksort là một trong những thuật toán hiệu quả nhất sử dụng phép so sánh, **độ phức tạp thời gian trung bình**.
- Hoạt động tốt trên nhiều loại dữ liệu và có thể được tối ưu hóa bằng cách chọn pivot thông minh hoặc kết hợp với các thuật toán khác cho dãy nhỏ.
- Không cần bộ nhớ phụ ngoài ngăn xếp đệ quy, giúp tiết kiệm không gian so với các thuật toán khác.

3.7 Sắp xếp theo cây/ Sắp xếp vun đống – Heap Sort

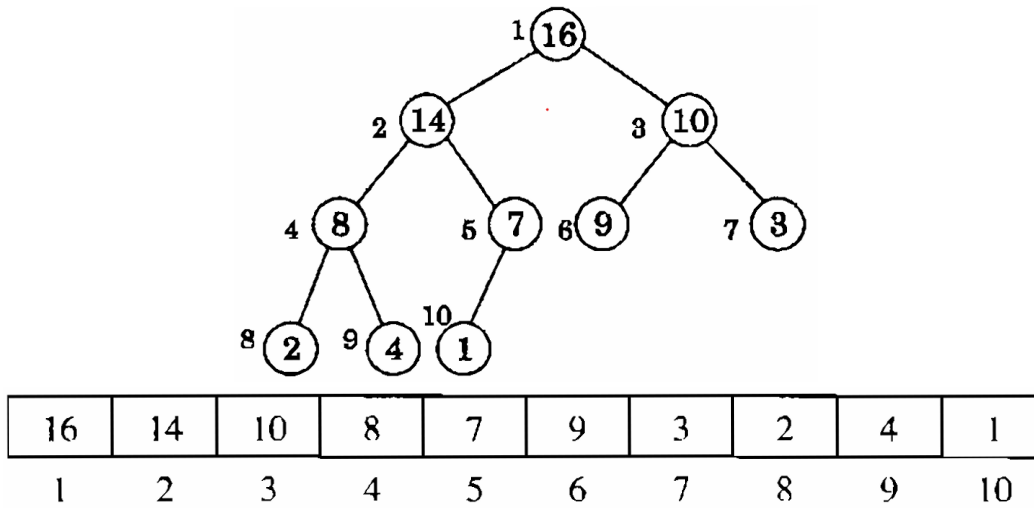
- **Ý tưởng của giải thuật Heap Sort:**

- Sắp xếp thông qua việc tạo các heap (đống) từ dãy ban đầu
- Heap là 1 **cây nhị phân hoàn chỉnh (complete binary tree)**, mỗi nút được gán một giá trị khóa sao cho thỏa điều kiện của mỗi loại cây dưới đây.
 - **Max heap (đối với sắp xếp tăng dần):** khóa của mỗi nút sẽ luôn **lớn hơn hoặc bằng khóa của các con** của nó (nếu có).
 - **Min heap (đối với sắp xếp giảm dần):** khóa của mỗi nút sẽ luôn **nhỏ hơn hoặc bằng khóa của các con** của nó (nếu có).
- **Cây nhị phân hoàn chỉnh (complete binary tree)**
 - Tất cả các mức của cây, trừ mức cuối cùng, được lấp đầy hoàn toàn (**mỗi nút ở các mức này có đúng 2 con**, trừ trường hợp nút lá).
 - **Ở mức cuối cùng, tất cả các nút được đặt càng sát về bên trái càng tốt** (tức là các nút được thêm từ trái sang phải mà không để lại khoảng trống).



complete tree with $N = 16$ nodes (height = 4)

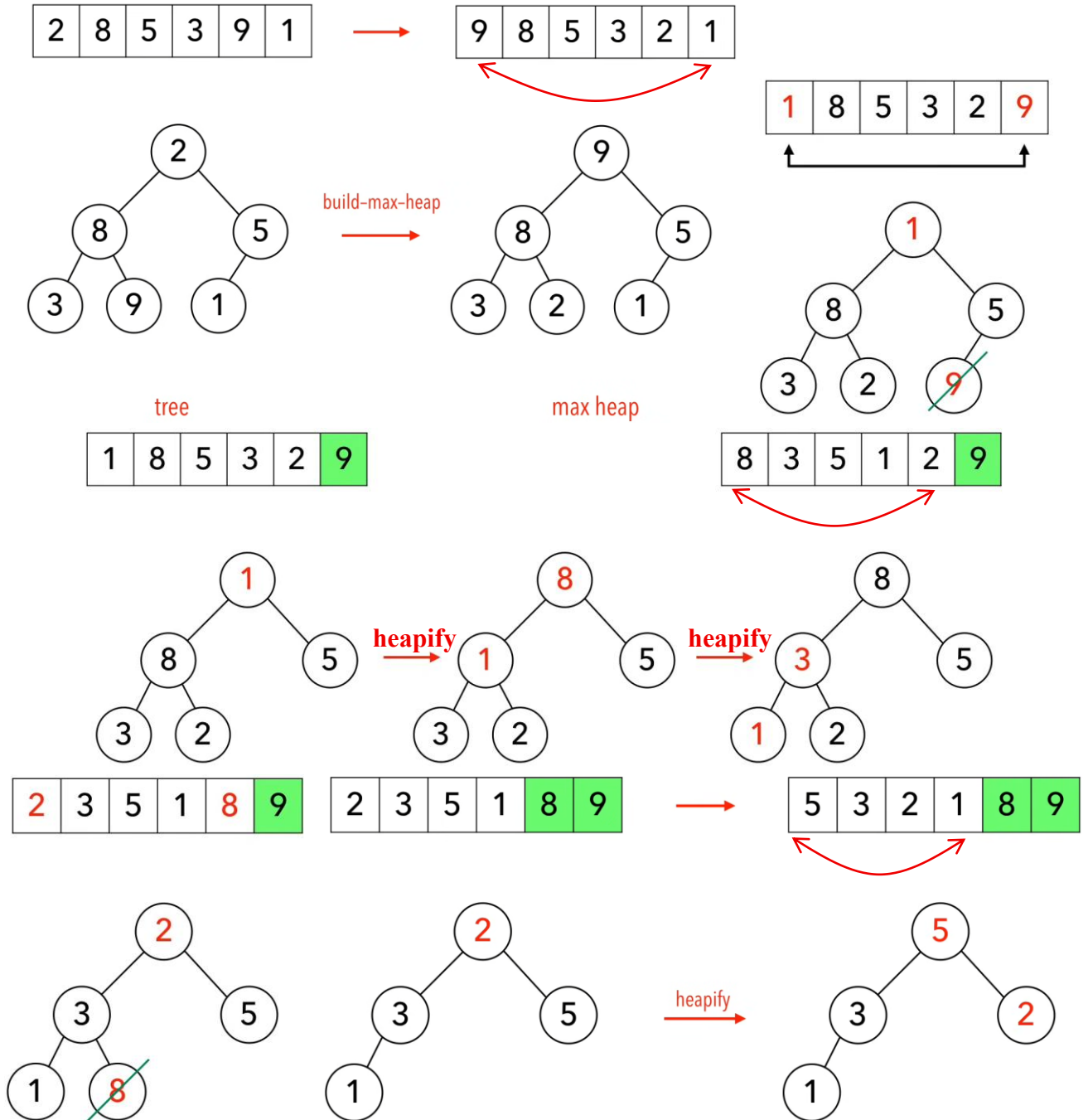
- Binary Heap thường được biểu diễn bởi một **mảng** (array-based binary heap) thay vì một cây nhị phân (pointer-based binary heap).



- Gốc của cây A là A[0]. Gọi *i là chỉ số của một node bất kỳ trong cây*, ta ký hiệu sau:
 - Chỉ số của node cha của i gọi là: $PARENT(i) = \lfloor (i-1)/2 \rfloor$
 - Chỉ số của node con trái của i gọi là: $LEFT(i) = 2i+1$
 - Chỉ số của node con phải của i gọi là: $RIGHT(i) = 2i+2$
 - Nút gốc ở chỉ số [0]
- Array-based binary heap phải thỏa mãn tính chất heap (heap property):
 - **Với Binary Max-Heap:** Tất cả các node i (không kể node gốc) thỏa $A[i] \leq A[PARENT(i)]$.
 - **Với Binary Min-Heap:** Tất cả các node i (không kể node gốc) thỏa $A[i] \geq A[PARENT(i)]$.
- **Các bước thực hiện:** Việc thực hiện giải thuật được chia thành 2 giai đoạn:
 - **Giai đoạn 1:** Tạo heap từ dãy ban đầu, **khi đó nút gốc của heap là phần tử lớn nhất (build-heap)**.
 - BUILD-HEAP thực hiện chuyển đổi dãy A[0... n-1] thành dãy có tính chất heap (array-based binary heap), với n là số lượng phần tử của A. Quá trình này sẽ sử dụng HEAPIFY trên các phần tử của mảng. Trong đó:
 - Các phần tử từ $A[n/2] \rightarrow A[n-1]$ là các node lá của cây nên **không cần xét**.
 - BUILD-HEAP sẽ thực hiện **HEAPIFY** tất cả các phần tử còn lại, **bắt đầu từ dưới lên trên (bottom-up manner)**, nghĩa là **từ A[n/2-1] ngược lên A[0]**.
 - **Heapify** là một hàm hoặc quy trình được sử dụng để đảm bảo rằng một cây nhị phân (hoặc một phần của cây) thỏa mãn tính chất heap.
 - Heapify khởi đầu từ một nút cụ thể (thường là nút cha), Heapify kiểm tra và điều chỉnh cây con (subtree) gốc tại nút đó để đảm bảo tính chất heap được duy trì. Nếu nút cha vi phạm tính chất heap (ví dụ, nhỏ hơn con trong max-heap), nó sẽ được đổi chỗ với nút con có giá trị lớn nhất (trong max-heap) hoặc nhỏ nhất (trong min-heap), sau đó tiếp tục kiểm tra cây con bị ảnh hưởng.
 - **Cách hoạt động của Heapify** (cho max-heap):
 - Bắt đầu tại một nút (chỉ số i).
 - So sánh giá trị của nút i với các nút con (chỉ số 2i+1 và 2i+2, nếu tính từ 0).
 - **Nếu nút cha nhỏ hơn nút con lớn nhất, đổi chỗ nút cha với nút con lớn nhất đó.**
 - Lặp lại quá trình này trên cây con bị ảnh hưởng (nút con vừa được đổi chỗ) cho đến khi tính chất heap được khôi phục.

- Nút gốc là nút có giá trị lớn nhất-nằm ở đầu mảng heap, ta *hoán vị phần tử cuối cùng của mảng heap với phần tử đầu tiên của heap*.
- Loại phần tử lớn nhất khỏi mảng heap.
- Hiệu chỉnh phần còn lại thành heap (**heapify**). Xét dãy heap mà đã loại bỏ phần tử cuối cùng. Phần tử gốc $A[0]$ của dãy heap hiện tại sẽ không còn giữ tính chất heap. Vì vậy ta cần **HEAPIFY** lại phần tử $A[0]$ để đưa dãy hiện tại về dãy có tính chất heap.
- Lặp lại quá trình cho tới khi heap chỉ còn 1 nút.

- **Ví dụ:**



- (2):
 - **largest != i**: Nếu largest không phải là nút cha (i), tức là một trong hai nút con có giá trị lớn hơn nút cha, thì tính chất max-heap bị vi phạm.
 - **swap(arr[i], arr[largest])**: Đổi chỗ giá trị của nút cha (arr[i]) với giá trị của nút con lớn nhất (arr[largest]), để đảm bảo nút cha có giá trị lớn nhất trong cây con.
 - **heapify(arr, n, largest)**: Sau khi đổi chỗ, cây con tại chỉ số largest (nơi nút cha ban đầu được chuyển xuống) có thể không còn thỏa mãn tính chất max-heap. Do đó, gọi đệ quy heapify trên nút largest để tiếp tục kiểm tra và khôi phục tính chất max-heap cho cây con bị ảnh hưởng.
 - Nếu **largest == i**, tức là nút cha đã có giá trị lớn nhất, không cần đổi chỗ hay gọi đệ quy, hàm kết thúc.

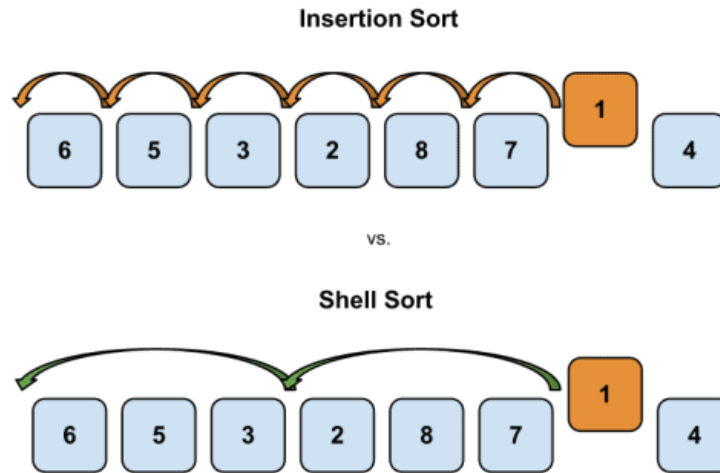
- **Thuật toán Heap Sort**

```
void heapSort(int arr[], int n) {
    //Xây dựng mảng có tính chất max-heap
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    //Hoán vị phần tử đầu và cuối, re-heap mảng sau khi không xét phần tử cuối nữa
    for (int i = n - 1; i >= 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

- **Độ phức tạp thuật toán:**
 - The HEAPSORT procedure takes time **$O(n \log n)$** , since the call to BUILDHEAP takes time **$O(n)$** and each of the $n-1$ calls to HEAPIFY takes time **$O(\log n)$** .
 - Heap Sort không nhạy cảm với dữ liệu đầu vào. Nó luôn thực hiện đúng một khối lượng công việc gần như cố định.

3.8 Shell Sort

- **Ý tưởng:**
 - Phân hoạch dãy thành các dãy con, sắp xếp các dãy con theo phương pháp chèn trực tiếp, dần dần sắp xếp cả dãy.
 - Thay vì so sánh và đổi chỗ các phần tử liền kề như Insertion Sort, Shell Sort so sánh và đổi chỗ các phần tử có khoảng cách xa nhau hơn, cách nhau một khoảng gap.
 - Sau đó khoảng cách gap được giảm dần qua các lần lặp (thường chia đôi hoặc theo một chuỗi khác), cuối cùng gap = 1 tương đương với chạy Insertion Sort trên toàn mảng.
 - Điều này giúp các phần tử di chuyển nhanh hơn đến vị trí gần đúng, giảm số lần đổi chỗ so với Insertion Sort.



- **Cách thực hiện**

- Phân chia dãy ban đầu thành những dãy con gồm các phần tử ở cách nhau **h vị trí**;
- Dãy ban đầu : $a[0], a[1], a[2], \dots, a[n]$ được xem như sự xen kẽ của các dãy con sau:
 - Dãy con thứ nhất: $a[0], a[h], a[2h], \dots$
 - Dãy con thứ hai: $a[1], a[h+1], a[2h+1], \dots$
 - Dãy con thứ ba: $a[2], a[h+2], a[2h+2], \dots$
 -
 - Dãy con thứ h : $a[h-1], a[2h-1], a[3h-1], \dots$
- Tiến hành **sắp xếp các phần tử trong cùng dãy con** bằng cách sử dụng phương pháp chèn trực tiếp (Insertion Sort), sẽ làm cho các phần tử được đưa về vị trí đúng tương đối ;
- Giảm khoảng cách h để tạo thành các dãy con mới ;
- **Dừng khi $h=1$**

- **Thuật toán sẽ**

- Sử dụng một **1 dãy $\{h_1, h_2, \dots, h_i\}$ là một dãy giảm dần và có $h_i = 1$** . Vấn đề là lựa chọn dãy h_i như thế nào cho hiệu quả
- Các chiến lược cho dãy h_i như sau (nếu thực hiện tính toán ra kết quả h_i là số thập phân thì lấy phần nguyên của kết quả):

- ▶ Shell đề xuất

$$h_1 = \frac{n}{2}, h_{i+1} = \frac{h_i}{2}$$

- ▶ Hibbard đề nghị

$$h_i = 2^i - 1$$

- ▶ Knuth đưa ra

$$h_1 = 1, h_{i+1} = 3h_i + 1$$

- ▶ Pratt đề xuất

$$h_1 = 1, h_i = 2^p 3^q$$

- Cài đặt thuật toán

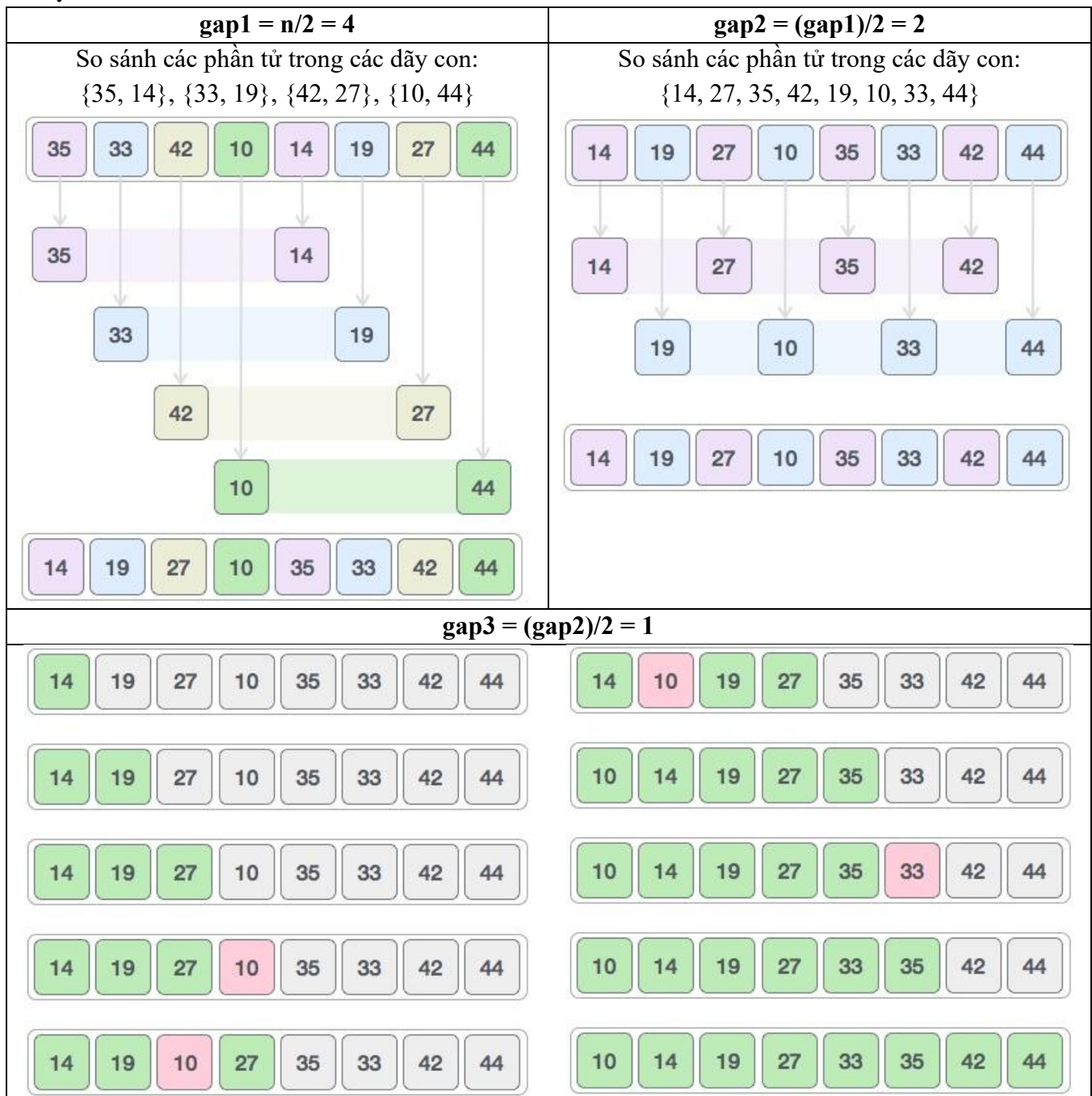
- Cách 1:

```
function shellSort(a) {
    let n = a.length;
    for (let gap = Math.floor(n / 2); gap >= 1; gap = Math.floor(gap / 2)) {
        for (let j = gap; j < n; j++) {
            for (let i = j - gap; i >= 0; i -= gap) {
                if (a[i + gap] > a[i]) {
                    break;
                } else {
                    let temp = a[i + gap];
                    a[i + gap] = a[i];
                    a[i] = temp;
                }
            }
        }
    }
    return a;
}
```

- Cách 2:

```
void shellSort(int a[], int n) {
    // Bắt đầu với gap là n/2
    for (int gap = n / 2; gap >= 1; gap /= 2) {
        // Áp dụng Insertion Sort trên các dãy con cách nhau gap
        for (int i = gap; i < n; i++) {
            int numberToInsert = a[i]; // Lưu phần tử cần chèn
            int pos = i - gap; // Vị trí để so sánh trong dãy con
            // Dịch các phần tử lớn hơn numberToInsert về sau trong dãy con
            while (pos >= 0 && a[pos] > numberToInsert) {
                a[pos + gap] = a[pos];
                pos -= gap;
            }
            a[pos + gap] = numberToInsert; // Chèn phần tử vào vị trí đúng
        }
    }
}
```

• Ví dụ:



- **Độ phức tạp thuật toán:** Shell Sort cũng như Insertion Sort là những giải thuật không có độ ổn định (Stability) vì nó phụ thuộc lớn vào dữ liệu đầu vào, thời gian chạy có sự chênh lệch lớn giữa trường hợp dữ liệu tốt (Best case) và trường hợp dữ liệu không phù hợp (Worst case). Cụ thể về độ phức tạp thời gian:
 - **Best: $O(n \log n)$** , vì có Số lượng gap: $O(\log n)$, Khi mảng đã sắp xếp hoặc gần sắp xếp, vòng while ít chạy,
 - **Worst: $O(n^2)$** , Khi mảng ngược hoàn toàn, cần dịch tối đa các phần tử trong mỗi dãy con.
 - **Average: $O(n^{1.25})$**

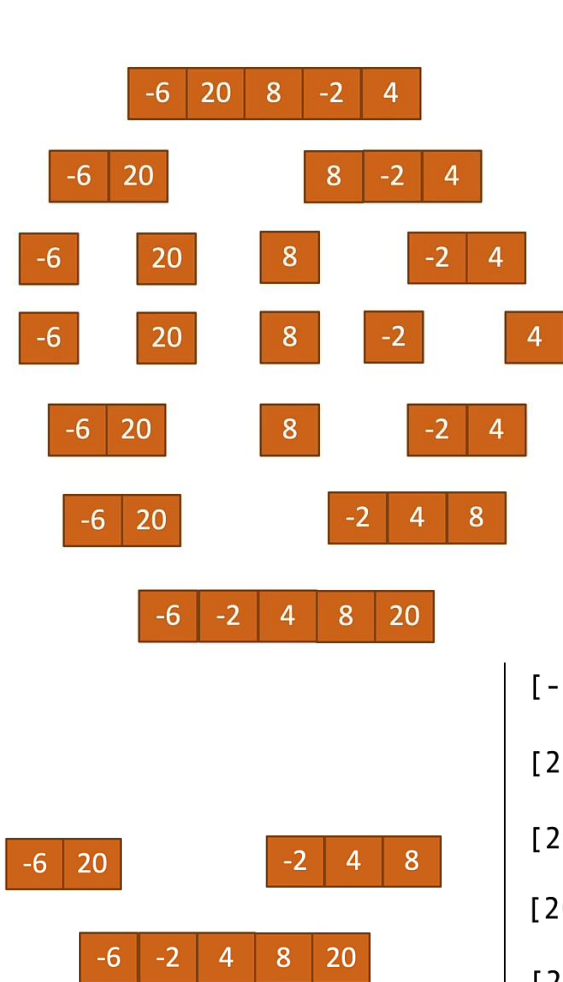
• **Tham khảo:** <https://www.youtube.com/watch?v=9crZRd8GPWM>

3.9. Trộn trực tiếp – Merge Sort

- Ý tưởng: Trộn 2 dãy đã được sắp xếp thành 1 dãy mới cũng được sắp xếp.
- Có 2 cách tiếp cận:

Top-down Merge Sort	Bottom-up Merge Sort
❖ Chia danh sách cần sắp xếp thành các danh sách con độc lập. Điều này được thực hiện bằng cách chia đôi danh sách ban đầu thành hai nửa (trái và phải) và tiếp tục chia đôi cho đến khi chỉ còn một phần tử trong mỗi danh sách con. ❖ Gộp/Trộn (merge) các mảng con đã chia, sắp xếp chúng trong quá trình gộp. ❖ Quá trình trộn này được thực hiện bằng cách so sánh phần tử đầu tiên của mỗi danh sách con và chọn phần tử nhỏ hơn để đưa vào danh sách kết quả. Tiếp tục thực hiện cho đến khi một trong hai danh sách con đã được trộn hết. ❖ Trả về danh sách đã được sắp xếp.	❖ Đầu tiên ta coi mỗi phần tử của dãy là 1 danh sách con gồm 1 phần tử (danh sách 1 phần tử là danh sách có thứ tự - đã được sắp). ❖ Tiếp theo tiến hành trộn từng cặp 2 dãy con 1 phần tử kề nhau để tạo thành các dãy con 2 phần tử được sắp. ❖ Các dãy con 2 phần tử được sắp này lại được trộn với nhau tạo thành dãy con 4 phần tử được sắp (size gấp đôi). Quá trình tiếp tục đến khi chỉ còn 1 dãy con duy nhất được sắp, đó chính là dãy ban đầu.
❖ Người ta thường dùng Top-down Merge Sort hơn vì nó dễ triển khai, tối ưu hóa tốt trong bộ nhớ cache, và được tích hợp sẵn trong nhiều thư viện lập trình. ❖ Bottom-up chỉ được dùng trong các trường hợp đặc biệt (file lớn, không dùng đệ quy); yêu cầu quản lý kích thước mảng con và vòng lặp phức tạp; không tận dụng tốt bộ nhớ cache do gộp không theo thứ tự tuần tự như top-down, không phổ biến bằng.	

- Ví dụ hình ảnh:



Mảng con KQ

Left Right

[-6] [20] => [] Left & right are not empty. $-6 < 20$

[] [20] => [-6] Left is empty. Push right array

[] [] => [-6, 20]

[8] [-2, 4] => [] Left & right are not empty. $-2 < 8$

[8] [4] => [-2] Left and right are not empty. $4 < 8$

[8] [] => [-2, 4] Right is empty. Push left array

[] [] => [-2, 4, 8]

So sánh 2 phần tử đầu tiên của 2 mảng con

[-6, 20] [-2, 4, 8] => [] Left & right are not empty. $-6 < -2$

[20] [-2, 4, 8] => [-6] Left & right are not empty. $-2 < 20$

[20] [4, 8] => [-6, -2] Left & right are not empty. $4 < 20$

[20] [8] => [-6, -2, 4] Left & right are not empty. $8 < 20$

[20] [] => [-6, -2, 4, 8] Right is empty. Push left array

[] [] => [-6, -2, 4, 8, 20]

- Cài đặt thuật toán

```
function mergeSort(arr){
  if(arr.length < 2)
    return arr;
  //Chia đôi mảng
  const mid = Math.floor(arr.length/2);
  const leftArr = arr.slice(0,mid);
  const rightArr = arr.slice(mid);
  return merge(mergeSort(leftArr), mergeSort(rightArr));
  // Gọi đệ quy mergeSort trên hai mảng con (leftArr và rightArr) để sắp xếp chúng.
  // Sau đó, gọi hàm merge để gộp hai mảng con đã sắp xếp thành một mảng lớn hơn.
  // Quá trình đệ quy tiếp tục cho đến khi các mảng con chỉ còn 1 phần tử, rồi gộp dần lên.
}
```



```

function merge(leftArr, rightArr){
  //Tạo mảng chứa KQ mảng con đã sắp xếp
  const sortedArr = [];
  while (leftArr.length && rightArr.length){
    if(leftArr[0] <= rightArr[0])
      sortedArr.push(leftArr.shift());
    else
      sortedArr.push(rightArr.shift());
  }
  return [...sortedArr, ...leftArr, ...rightArr];
  // Khi một trong hai mảng hết phần tử, vòng lặp dừng.
  // Nếu còn phần tử trong leftArr hoặc rightArr (do hai mảng có thể không đều độ dài),
  // thêm tất cả phần tử còn lại vào sortedArr.
  // Toán tử spread (...) được dùng để nối các mảng.
}

const arr = [-6,20,8,-2,4];
console.log(mergeSort(arr));

```

- o Ví dụ về code:

1) Lần gọi gốc

mergeSort([-6,20,8,-2,4])

- mid = Math.floor(5/2) = 2
- leftArr = [-6, 20]
- rightArr = [8, -2, 4]

Chia tiếp cho đến mảng kích thước 1.

2) Xử lý leftArr = [-6, 20]

mergeSort([-6,20]) → mid = 1 → chia thành [-6] và [20]

- merge([-6], [20]):
 - o so sánh -6 <= 20 → true → đưa -6 vào sortedArr ⇒ sortedArr = [-6]
 - o một bên mảng rỗng, ghép mảng còn lại vào sortedArr → kết quả: [-6, 20]

Kết quả trái: **leftArr = [-6, 20]**

3) Xử lý rightArr = [8, -2, 4]

mergeSort([8,-2,4]) → mid = 1 → left = [8], right = [-2, 4]

3.1) Xử lý right = [-2, -4]:

- mergeSort([-2,4]) → mid = 1 → [-2] và [4]
- merge([-2], [4]):
 - o so sánh -2 <= -4 → true → đưa -2 vào sortedArr → sortedArr = [-2]
 - o left rỗng? còn right = [4] → ghép phần còn lại vào sortedArr → kết quả: [-2, 4]

3.2) Merge [8] và [-2,4]

- merge([8], [-2,4]):
 - o so sánh 8 <= -2 → false → push -2 → sortedArr = [-2]
 - o so sánh 8 <= 4 → false → push 4 → sortedArr = [-2,4]

- right giờ rỗng → ghép phần còn lại [8] → kết quả: [-2,4,8]
- Kết quả phải: [-2, 4, 8]

4) Merge cuối cùng: merge([-6,20], [-2,4,8])

Ban đầu: left = [-6,20], right = [-2,4,8], sortedArr = []

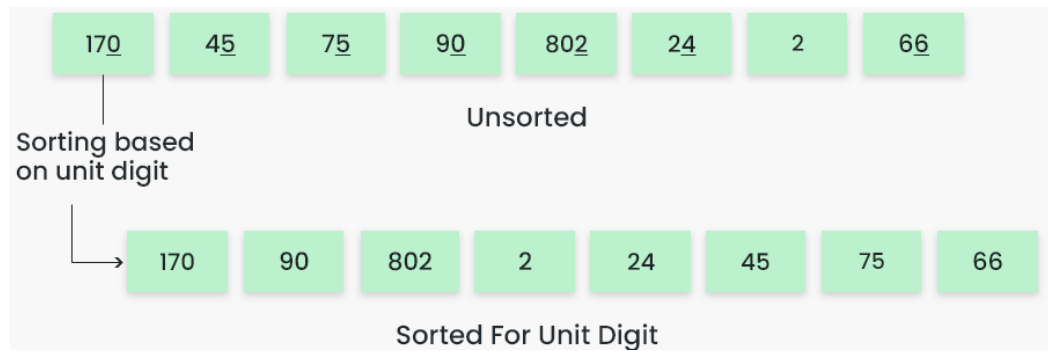
- so sánh -6 <= -2 → true → push -6 → sortedArr = [-6] → left = [20], right = [-2,4,8]
- so sánh 20 <= -2 → false → push -2 → sortedArr = [-6,-2] → left = [20], right = [4,8]
- so sánh 20 <= 4 → false → push 4 → sortedArr = [-6,-2,4] → left = [20], right = [8]
- so sánh 20 <= 8 → false → push 8 → sortedArr = [-6,-2,4,8] → left = [20], right = []
- ghép phần còn lại [20]

Kết quả cuối: [-6, -4, -2, 8, 20]

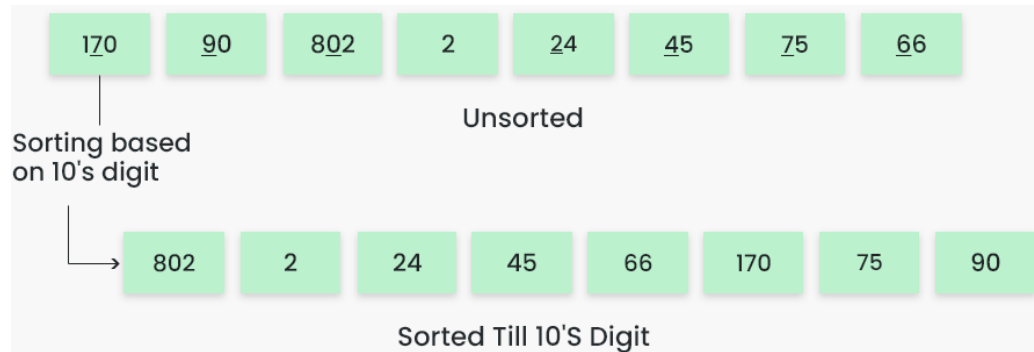
- **Độ phức tạp thuật toán:**
 - **Thời gian: $O(n \log n)$** (do chia mảng thành $\log n$ mức và gộp với chi phí $O(n)$).
 - **Không gian: $O(n)$** (do cần mảng phụ để lưu kết quả khi gộp).
- **Đánh giá**
 - Thích hợp cho các danh sách truy xuất tuần tự (file, danh sách đơn).
 - Mergesort *requires an extra array whose size equals to the size of the original array.*
 - If we use a linked list, we do not need an extra array
 - Có thể thực hiện sắp xếp mà không cần nạp toàn bộ danh sách lên RAM (**External Sorting**).
 - Trường hợp danh sách đã có những đoạn con có thứ tự → Trộn tự nhiên (**Natural Merge Sort**).

3.10 Sắp xếp theo cơ số - Radix Sort

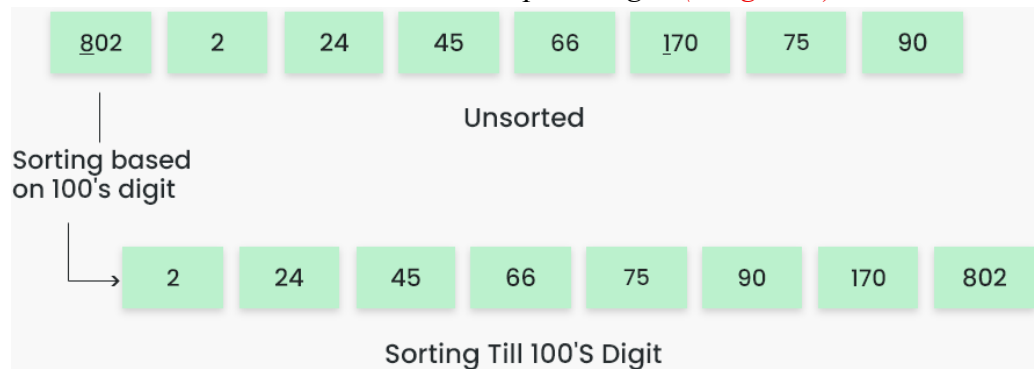
- Radix Sort là một thuật toán tiếp cận theo một hướng hoàn toàn khác.
- Nếu như trong các thuật toán khác, cơ sở để sắp xếp luôn là việc *so sánh giá trị* của 2 phần tử thì Radix Sort lại dựa trên *nguyên tắc phân loại thư của bưu điện*. Vì lý do đó Radix Sort còn có tên là **Postman's Sort**.
- Radix Sort không hề quan tâm đến việc so sánh giá trị của phần tử mà bản thân *việc phân loại và trình tự phân loại* sẽ tạo ra thứ tự cho các phần tử.
- Mô phỏng lại qui trình trên, để sắp xếp dãy $a_1, a_2, \dots, a_n$, giải thuật Radix Sort thực hiện như sau:
 - Trước tiên, ta có thể **giả sử mỗi phần tử a_i trong dãy $a_1, a_2, \dots, a_n$ là một số nguyên có tối đa m chữ số**.
 - Ta phân loại các phần tử lần lượt theo các chữ số **hàng đơn vị, hàng chục, hàng trăm, ...** tương tự việc phân loại thư theo tỉnh thành, quận huyện, phường xã,
- **Ví dụ 1:** *To perform radix sort on the array [170, 45, 75, 90, 802, 24, 2, 66], we follow these steps:*
 - **Step 1:** *Find the largest element, which is 802. It has three digits, so we will **iterate three times (số lượng chữ số tối đa cần so sánh là 3 chữ số, gồm hàng đơn vị, hàng chục, hàng trăm)**.*
Ta có thể coi các chữ số < 100 sẽ được lấp đầy các số 0 ở các vị trí mà nó thiếu, như thế ta có thể viết lại mảng như sau: [170, 045, 075, 090, 802, 024, 002, 066].
 - **Step 2:** *Sort the elements based on the **unit** place digits (hàng đơn vị - nghĩa là ta đi so sánh các phần tử thông qua chữ số ở hàng đơn vị của mỗi phần tử, sau đó sắp xếp tăng dần cũng chỉ theo hàng đơn vị) ($X=0$).*



- **Step 3:** Sort the elements based on the **tens** place digits (*hàng chục*).



- **Step 4:** Sort the elements based on the **hundreds** place digits (*hàng trăm*).



- **The array is now sorted in ascending order.**

- **Ví dụ 2:** Sắp xếp dãy sau tăng dần theo giải thuật sắp xếp cơ số Radix Sort:

7013, 8425, 1239, 428, 1424, 7009, 4518, 3252, 9170, 999, 1725, 701.

12	070 <u>1</u>										
11	172 <u>5</u>										
10	099 <u>9</u>										
9	917 <u>0</u>										
8	325 <u>2</u>										
7	451 <u>8</u>										
6	700 <u>9</u>										
5	142 <u>4</u>										
4	042 <u>8</u>										
3	123 <u>9</u>										
2	842 <u>5</u>										
1	701 <u>3</u>										
CS	A	0	1	2	3	4	5	6	7	8	9

- Cột dọc: Các phần tử của mảng A, chỉ số từ 1 đến 12.
- Cột ngang: Các chữ số, từ 0 đến 9, dùng để sắp xếp các phần tử khi ta xét ở một vị trí của các phần tử (hàng đơn vị, hàng chục...).

○ **Phân loại hàng đơn vị**

12											
11											
10											
9											
8											
7											
6											
5											
4											
3											099 <u>9</u>
2							172 <u>5</u>			451 <u>8</u>	700 <u>9</u>
1		917 <u>0</u>	070 <u>1</u>	325 <u>2</u>	701 <u>3</u>	142 <u>4</u>	842 <u>5</u>			042 <u>8</u>	123 <u>9</u>
CS	A	0	1	2	3	4	5	6	7	8	9

- Xét hàng đơn vị của các phần tử, phân loại chúng vào từng cột tương ứng với từng chữ số.
- Sau đó sắp xếp tăng dần các phần tử dựa theo hàng đơn vị. Chẳng hạn 9170 sẽ được đưa vào mảng A trước, sau đó đến 0701,...

12	09 <u>9</u> 9										
11	70 <u>0</u> 9										
10	12 <u>3</u> 9										
9	45 <u>1</u> 8										
8	04 <u>2</u> 8										
7	17 <u>2</u> 5										
6	84 <u>2</u> 5										
5	14 <u>2</u> 4										
4	70 <u>1</u> 3										
3	32 <u>5</u> 2										
2	07 <u>0</u> 1										
1	91 <u>7</u> 0										
CS	A	0	1	2	3	4	5	6	7	8	9

○ Phân loại hàng chục

12											
11											
10											
9											
8											
7											
6											
5											
4				04 <u>2</u> 8							
3				17 <u>2</u> 5							
2		70 <u>0</u> 9	45 <u>1</u> 8	84 <u>2</u> 5							
1		07 <u>0</u> 1	70 <u>1</u> 3	14 <u>2</u> 4	12 <u>3</u> 9		32 <u>5</u> 2		91 <u>7</u> 0		09 <u>9</u> 9
CS	A	0	1	2	3	4	5	6	7	8	9

12	0 <u>9</u> 99										
11	9 <u>1</u> 70										
10	3 <u>2</u> 52										
9	1 <u>2</u> 39										
8	0 <u>4</u> 28										
7	1 <u>7</u> 25										
6	8 <u>4</u> 25										
5	1 <u>4</u> 24										
4	4 <u>5</u> 18										
3	7 <u>0</u> 13										
2	7 <u>0</u> 09										
1	0 <u>7</u> 01										
CS	A	0	1	2	3	4	5	6	7	8	9

○ Phân loại hàng trăm

12											
11											
10											
9											
8											
7											
6											
5											
4											
3						0 <u>4</u> 28					
2		7 <u>0</u> 13		3 <u>2</u> 52		8 <u>4</u> 25			1 <u>7</u> 25		
1		7 <u>0</u> 09	9 <u>1</u> 70	1 <u>2</u> 39		1 <u>4</u> 24	4 <u>5</u> 18		0 <u>7</u> 01		0 <u>9</u> 99
CS	A	0	1	2	3	4	5	6	7	8	9

12	<u>0</u> 999										
11	<u>1</u> 725										
10	<u>0</u> 701										
9	<u>4</u> 518										
8	<u>0</u> 428										
7	<u>8</u> 425										
6	<u>1</u> 424										
5	<u>3</u> 252										
4	<u>1</u> 239										
3	<u>9</u> 170										
2	<u>7</u> 013										
1	<u>7</u> 009										
CS	A	0	1	2	3	4	5	6	7	8	9

○ Phân loại hàng nghìn

12											
11											
10											
9											
8											
7											
6											
5											
4											
3		<u>0</u> 999	<u>1</u> 725								
2		<u>0</u> 701	<u>1</u> 424						<u>7</u> 013		
1		<u>0</u> 428	<u>1</u> 239		<u>3</u> 252	<u>4</u> 518			<u>7</u> 009	<u>8</u> 425	<u>9</u> 170
CS	A	0	1	2	3	4	5	6	7	8	9

12	9170										
11	8425										
10	7013										
9	7009										
8	4518										
7	3252										
6	1725										
5	1424										
4	1239										
3	0999										
2	0701										
1	0428										
CS	A	0	1	2	3	4	5	6	7	8	9

- Kết quả cuối cùng dãy đã được sắp xếp tăng dần:

428, 701, 999, 1239, 1424, 1725, 3252, 4518, 7009, 7013, 8425, 9170

3.11. Shaker Sort

- Shaker Sort là một cải tiến của Bubble Sort.
- Sau khi đưa phần tử nhỏ nhất về đầu mảng sẽ đưa phần tử lớn nhất về cuối dãy.
- Do đưa các phần tử về đúng vị trí ở cả hai đầu nên Shaker Sort sẽ giúp cải thiện thời gian sắp xếp dãy số do giảm được độ lớn của mảng đang xét ở lần so sánh kế tiếp.
- Tham khảo: <https://daynhauhoc.com/t/cau-truc-du-lieu-va-giai-thuat-giai-thuat-sap-xep-rung-lac-shaker-sort/126359>

3.12. Counting Sort

- Counting Sort là phương pháp sắp xếp sử dụng thông tin của khóa.
- Các số sẽ được đếm
- Căn cứ vào giá trị đếm thì sẽ xác định chính xác vị trí của số trong mảng
- Tham khảo: <https://ant.ncc.plus/counting-sort-hieu-ve-thuat-toan-sap-xep-dem/>